

深圳大学实验报告

课程名称： 编译原理

实验项目名称： 语法制导翻译和中间代码生成

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 王祎乐

报告人： 学号： 班级：

实验时间： 2026 年 6 月 12 日至 6 月 26 日

实验报告提交时间： 2026 年 6 月 19 日

教务处制

实验目的与要求:

目的: 通过设计、实现赋值等语句的中间代码生成程序, 熟练掌握语法制导翻译模式及其应用。

要求:

基于简单算术表达式的赋值语句的翻译与中间代码生成

对由多条简单算术表达式构成的赋值语句进行翻译, 将其生成对应的中间代码四元组序列。

输入文件: 由多条使用简单算术表达式形成的赋值语句组成的源代码, 语句间用“;” 隔开

目标: 对每条赋值语句中的算术表达式进行拆解和中间变量引入, 最终输出等价的三地址码 (或四元组) 序列。

输出文件: 四元组序列

例如: 输入文件内容: $a=b+c*e/g; d=f*g;$

则输出文件内容可以为:

- $(*, c, e, t1)$
- $(/, t1, g, t2)$
- $(+, b, t2, t3)$
- $(=, t3, 0, a)$
- $(*, f, g, t4)$
- $(=, t4, 0, d)$

其中, 临时变量 $t1, t2, t3, \dots$ 按出现顺序分配, 0 作为赋值运算的占位符。

注: 部分代码写完了, 填空即可

方法、步骤:

要完成本实验, 依据实验要求进行分解, 需要完成的实验步骤是:

i 理论设计: 定义赋值语句、四则算术表达式文法, 配套 S 属性语法制导翻译语义规则;

ii 程序分层模块化设计: 分为输入读取模块、语句分割模块、分词模块、中缀转后缀模块、四元组生成模块;

iii 功能拓展: 增加命令行传参交互逻辑, 支持动态选择 examples 目录下 1~10.txt 测试用例;

iv 编码实现: 按照文法与语义规则补全 C++ 代码模板, 实现各子功能;

v 调试校验: 分别对分词、后缀转换、临时变量分配、四元组输出单独测试, 修复栈操作顺序、运算符优先级、文件读取边界异常;

vi 批量测试: 使用 1~10 全部 txt 文件遍历运行, 验证多语句、多优先级表达式、空白字符、无末尾分号等边界场景;

vii 结果比对: 核对程序输出四元组与人工推导中间代码是否一致, 完成实验验证。

1. 你为简单算术表达式、赋值语句、顺序结构语句设计的产生式和语义规则分别是什么?

(1) 文法产生式 (S 属性自下而上翻译文法)

顺序语句 (多条赋值语句以;分隔)

$Prog \rightarrow StmtSeq$

$StmtSeq \rightarrow StmtSeq S ; | S$

赋值语句

$S \rightarrow id = E$

算术表达式，区分加减、乘除优先级，左结合

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow id$

(2) 配套语法制导语义规则

① $Prog \rightarrow StmtSeq$

无操作，传递语句序列

② $StmtSeq \rightarrow StmtSeq_1 S ;$

依次执行 $StmtSeq_1$ 、 S 的语义动作，连续生成多条语句四元组

③ $StmtSeq \rightarrow S ;$

执行单条赋值语句 S 的翻译动作

④ $S \rightarrow id = E$

`emit( '=', E.place, 0, id.lex );`

语义：生成赋值四元组，将表达式 E 计算结果存入左侧变量 id ，无第二操作数用 0 占位。

⑤ $E \rightarrow E_1 + T$

`E.place = newtemp();`

`emit( '+', E1.place, T.place, E.place );`

语义：新建临时变量，生成加法运算四元组，保存运算结果地址。

⑥ $E \rightarrow E_1 - T$

`E.place = newtemp();`

`emit( '-', E1.place, T.place, E.place );`

⑦ $T \rightarrow T_1 * F$

`T.place = newtemp();`

`emit( '*', T1.place, F.place, T.place );`

⑧ $T \rightarrow T_1 / F$

`T.place = newtemp();`

`emit( '/', T1.place, F.place, T.place );`

⑨ $F \rightarrow id$

`F.place = id.lex;`

语义：操作数直接使用变量名作为地址，无需生成中间代码。

2. 如何对每条赋值语句进行“分左右”并拆出算术表达式？

(1) 语句预处理

原始文件全部文本按;分割，得到单条完整赋值语句，调用 `trim()` 清除语句首尾空格、制表符，过滤空语句。

(2) 分割左值与表达式右值

对单条语句字符串调用 `string::find('=')` 定位赋值符号下标：

① 下标前子串：去除空白后为左变量 LHS（赋值目标）；

② 下标后子串：去除空白后为算术表达式 RHS（待翻译的运算式）；

③ 未找到=的非法语句直接跳过，不参与翻译。

(3) 表达式分词拆分

对右侧算术表达式执行 `tokenize` 分词：

①遍历表达式字符，跳过空白；
②连续字母 / 数字读取为完整标识符变量；
③+ - * /单字符单独作为运算符 token；
最终得到表达式 token 列表，供给调度场算法转换后缀。

3. 如何根据后缀表达式生成四元组、管理并分配临时变量？

(1) 初始化空字符串栈，用于缓存变量 / 临时变量操作数；

(2) 按顺序遍历后缀表达式每一个 token：

若 token 为变量标识符：直接压入操作数栈；

若 token 为+ - * /运算符：

① 栈顶先弹出右操作数；

② 再次弹出栈顶左操作数；

③ 分配新临时变量 tN；

④ 打印四元组 (op, 左操作数, 右操作数, tN)；

⑤ 将临时变量 tN 压回栈，作为上层运算的操作数；

(3) 后缀遍历完成后，栈内仅剩余一个元素，即整条表达式最终结果临时变量；

(4) 输出赋值四元组 (=, 最终临时变量, 0, 左侧赋值变量)，完成单条语句翻译。

后缀表达式天然消除括号、自带运算顺序，栈结构保证二元运算操作数顺序正确；全局连续临时变量统一管理所有中间运算结果，符合四元组中间代码拆分复合表达式的规范。

实验过程及内容：

1.基础工具函数补全

按照模板标注的填空点位，依次实现辅助工具：

```
10 // 去除字符串两端空白
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
11 static string trim(const string &s) {
12     size_t i = 0, j = s.size();
13     while (i < s.size() && isspace((unsigned char)s[i])) i++;
14     while (j > 0 && isspace((unsigned char)s[j - 1])) j--;
15     return s.substr(i, j - i);
16 }
17
```

trim() 去空白函数：修复字符串首尾边界判断，避免空白字符干扰语句分割；

```
18 // 判断 token 是否为运算符
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
19 static bool isOperator(const string &tok) {
20     return (tok == "+" || tok == "-" || tok == "*" || tok == "/");
21 }
22
23 // 返回运算符优先级（数字越大优先级越高）
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
24 static int precedence(const string &op) {
25     if (op == "*" || op == "/") return 2;
26     if (op == "+" || op == "-") return 1;
27     return 0;
28 }
29
```

isOperator()、precedence()：区分运算符并定义优先级数值；

```

39         // 如果是字母或数字，读取完整标识符（变量名或常数）
40         if (isalnum((unsigned char)expr[i])) {
41             int j = i;
42             while (j < n && isalnum((unsigned char)expr[j])) j++;
43             tokens.push_back(expr.substr(i, j - i));
44             i = j;
45         }

```

tokenize() 分词函数：区分连续字母数字变量与单字符运算符，避免变量名被错误拆分。

2. 调度场中缀转后缀实现

```

62         // 如果栈顶也是运算符且优先级 >= 当前 tok，就弹出
63         while (!opStack.empty() && isOperator(opStack.top())
64             && precedence(opStack.top()) >= precedence(tok)) {
65             output.push_back(opStack.top());
66             opStack.pop();
67         }
68         opStack.push(tok);

```

搭建运算符栈与输出列表基础框架：

实现优先级弹出逻辑：栈顶运算符优先级 $\geq$ 当前运算符时持续出栈；

边界处理：遍历结束后清空栈内剩余运算符，保证后缀表达式完整。

3. 四元组生成逻辑编码

设计操作数栈缓存变量 / 临时变量：

```

90         string right = st.top(); st.pop();
91         string left  = st.top(); st.pop();

```

实现二元运算栈弹出逻辑：先弹右操作数，再弹左操作数（初期踩坑：颠倒顺序导致减法、除法结果错误，通过单条语句人工推演修正）：

```

92         tempCnt++;
93         string tvar = "t" + to_string(tempCnt);
94         cout << "(" << tok << ", " << left << ", " << right << ", " << tvar << ")\n";
95         st.push(tvar);

```

全局临时变量计数器 tempCnt，每次运算自增生成 tN，运算完成后将临时变量压回栈供上层使用：

```

103         string finalTemp = st.top();
104         cout << "(=, " << finalTemp << ", 0, " << lhsVar << ")\n";

```

遍历完成后输出赋值四元组，完成单条语句翻译。

4. 文件读取与语句分割编码

```

151         // 读取全部文件内容
152         string rawInput;
153         {
154             ostringstream oss;
155             string line;
156             while (getline(fin, line)) {
157                 oss << line;
158             }
159             rawInput = oss.str();
160         }
161         fin.close();

```

读取文件全部文本拼接为单行字符串；

```

186         size_t posEq = stmt.find('=');
187         if (posEq == string::npos) continue;
188         string lhs = trim(stmt.substr(0, posEq));
189         string rhs = trim(stmt.substr(posEq + 1));
190     }

```

以;分割多条语句，过滤空语句、纯空白语句；

通过 find(=)拆分每条赋值语句的左变量与右侧表达式，无=的非法语句直接跳过。

5.修改主函数入参

```

108 // 主函数增加 argc/argv 命令行参数接收
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
109 int main(int argc, char* argv[]) {

```

将 int main()修改为 int main(int argc, char* argv[])，接收终端传入的命令行参数；

参数分支逻辑编写

①传入 1 个数字参数。增加异常捕获，若参数为非数字直接报错退出；校验数字范围 1~10，超出则提示错误；合法参数拼接文件路径 examples/X.txt；

```

113     int selectNum = 0;
114     // 1. 判断是否传入命令行参数
115     if (argc == 2) {
116         try {
117             selectNum = stoi(argv[1]);
118         } catch (...) {
119             cerr << "错误：命令行参数必须是数字 1~10!\n";
120             return 1;
121         }
122         // 校验数字范围
123         if (selectNum < 1 || selectNum > 10) {
124             cerr << "错误：文件编号范围仅支持 1 ~ 10\n";
125             return 1;
126         }

```

②无传入参数。循环交互式输入文件编号，输入非法数字则重复提示，直到输入 1~10 合法数值；

```

127     } else {
128         // 2. 无参数时，交互式手动输入
129         cout << "==== 中间代码生成程序 =====\n";
130         cout << "使用方式1(推荐): ./ex5 [1-10] 例如 ./ex5 10\n";
131         cout << "使用方式2: 直接输入文件编号 1~10\n";
132         while (true) {
133             cout << "请输入要读取的txt文件编号:";
134             cin >> selectNum;
135             if (selectNum >= 1 && selectNum <= 10) break;
136             cout << "输入非法！只能输入 1~10 的整数，请重新输入\n\n";
137         }
138     }

```

③路径变量替换

```
140 // 拼接文件路径
141 string filePath = "examples/" + to_string(selectNum) + ".txt";
142 cout << "\n正在读取输入文件:" << filePath << "\n四元组输出:" << "\n\n";
143
144 // 打开目标文件
145 ifstream fin(filePath);
146 if (!fin.is_open()) {
147     cerr << "致命错误: 无法打开文件 " << filePath << ", 检查目录是否存在! \n";
148     return 1;
149 }
150
```

删除硬编码"examples/10.txt", 统一使用变量 filePath 存储拼接后的文件路径;

增加文件打开容错提示

文件不存在时打印清晰错误信息, 终止程序, 方便排查路径错误。

6. 运行结果

● → /workspace git:(master) X ./ex5 1	正在读取输入文件: examples/1.txt 四元组输出: (+, b, c, t1) (=, t1, 0, a)	● → /workspace git:(master) X ./ex5 5	正在读取输入文件: examples/5.txt 四元组输出: (*, a, b, t1) (*, t1, c, t2) (=, t2, 0, x)
● → /workspace git:(master) X ./ex5 2	正在读取输入文件: examples/2.txt 四元组输出: (*, b, c, t1) (+, t1, d, t2) (=, t2, 0, a)	● → /workspace git:(master) X ./ex5 6	正在读取输入文件: examples/6.txt 四元组输出: (*, b, c, t1) (+, a, t1, t2) (/, d, e, t3) (-, t2, t3, t4) (=, t4, 0, result)
● → /workspace git:(master) X ./ex5 3	正在读取输入文件: examples/3.txt 四元组输出: (*, c, d, t1) (+, b, t1, t2) (=, t2, 0, a)	● → /workspace git:(master) X ./ex5 7	正在读取输入文件: examples/7.txt 四元组输出: (/, x, y, t1) (*, z, w, t2) (+, t1, t2, t3) (=, t3, 0, m)
● → /workspace git:(master) X ./ex5 4	正在读取输入文件: examples/4.txt 四元组输出: (+, a, b, t1) (+, t1, c, t2) (=, t2, 0, x)	● → /workspace git:(master) X ./ex5 8	正在读取输入文件: examples/8.txt 四元组输出: (-, b, c, t1) (-, t1, d, t2) (=, t2, 0, a)

```
● → /workspace git:(master) X ./ex5 9
```

正在读取输入文件: examples/9.txt

四元组输出:

```
(/, b, c, t1)
(/, t1, d, t2)
(=, t2, 0, a)
```

```
● → /workspace git:(master) X ./ex5 10
```

正在读取输入文件: examples/10.txt

四元组输出:

```
(+, b, c, t1)
(=, t1, 0, a)
(*, e, f, t2)
(+, t2, g, t3)
(=, t3, 0, d)
(*, j, k, t4)
(/, t4, l, t5)
(-, i, t5, t6)
(=, t6, 0, h)
```

实验结论:

本次实验程序对 examples/1.txt ~ 10.txt 全部测试用例完成批量运行, 所有用例均可正常读取、解析并输出规范四元组中间代码, 整体功能完全满足实验文档要求, 验证结果如下:

1.输入兼容能力达标

支持命令行传参 ./ex5 [1-10] 直接指定测试文件, 无参数时提供交互式手动输入;

兼容单条赋值语句、多条连续赋值语句 (如 10.txt 包含 3 条独立赋值语句);

自动过滤空行、多余空白字符、无末尾分号等边界输入, 语句分割逻辑稳定无崩溃。

2.运算优先级与结合性处理正确

程序通过调度场算法实现 * / 优先级高于 + -, 四则运算符全部左结合, 输出四元组运算顺序与数学运算规则完全匹配:

混合优先级示例 (6.txt, 表达式 $a + b * c - d / e$): 先计算 $b * c$ 、 d / e , 再依次执行加法、减法, 输出顺序 $(*, b, c, t1) \rightarrow (+, a, t1, t2) \rightarrow (/ , d, e, t3) \rightarrow (-, t2, t3, t4)$, 符合先乘除后加减规则;

同级连续运算示例 (4.txt $a+b+c$ 、8.txt $b-c-d$ 、9.txt $b/c/d$): 同级运算符从左至右依次计算, 临时变量 t1 结果作为下一次运算左操作数, 严格遵循左结合规则。

3.临时变量分配机制符合规范

全局计数器统一管理临时变量 t1、t2、t3……, 多条语句连续运行时编号持续递增, 不会重置; 每个二元算术运算自动分配唯一临时变量存储中间结果, 完美拆分复合表达式, 将复杂运算拆解为标准二元四元组。

4.赋值语句语义翻译准确

赋值操作使用独立操作码=, 四元组格式固定为(=, 表达式最终临时变量, 0, 左值变量), 第二个参数 0 作为无意义占位符, 与实验给定语法制导语义规则完全对应, 清晰区分算术运算与存储赋值两类操作, 不存在运算符混淆错误。

心得体会：

除了实验的心得体会外，还要额外增加说明你对语法制导翻译和中间代码生成程序的理解。
本次实验完整复现了语义分析 + 中间代码生成核心流程，从字符串原始输入到最终四元组输出，每一步都对应编译原理理论知识点。调试过程中发现栈操作顺序、运算符优先级是最容易出错的环节，也加深了对逆波兰表达式优势的理解：后缀表达式无需括号，仅依靠栈即可完成运算顺序解析，非常适合机器生成中间代码。模块化编程思路大幅降低调试成本，分词、转换、代码生成分离，可单独测试每个模块功能。

语法制导翻译（SDT）

基于属性文法，为每条语法产生式附加语义动作，在语法分析归约时同步执行代码生成。本实验文法是 S 属性文法，仅使用综合属性，从叶子节点向上传递表达式计算结果地址，适合自底向上分析。语义动作本质是封装临时变量创建、四元组输出等操作，把语法结构映射为中间代码。

中间代码（四元组）作用

四元组属于独立于机器的中间表示，剥离了高级语言语法细节，保留运算逻辑；不依赖 CPU 架构，可用于优化、解释执行或后端目标代码生成。使用临时变量拆分复合表达式，将复杂运算拆解为二元运算，简化后续优化与代码生成。

整体流程逻辑

词法分析分词→语法分析（调度场模拟归约）→语法制导语义动作生成中间代码。算术表达式翻译核心是按优先级拆分子表达式、用临时变量保存中间结果，所有二元运算统一转为标准四元组，赋值语句作为特殊运算完成变量存储。

指导教师批阅意见：

成绩评定：

指导教师签字：
年 月 日

备注：